

•



- [Deploying a project into Production](#)
- [Designing a workflow](#)
- Adding plans

On this page

Adding plans

Was this page helpful? 

Why

One of the great advantages of Pulse is to make the deployment of Data Science work into runtime seamless and without any significant effort. **Workflow Plans** are the direct translation of deployable [Data Science Plans](#) as runtime services.

Workflow Plans thus allow you to process live data and generate features exactly the same way as during the Data Science stage of the project.

Once a deployable **Data Science Plan**, for example, a features plan, is tested and ready to deploy to Production, Pulse creates an execution. A **Plan Execution** represents a snapshot of the **Plan** and all of its configurations at the time it ran, making it available for runtime.

How to

To add and configure a **Plan** to your **Workflow**, for example, to calculate features on live data, execute the following actions:

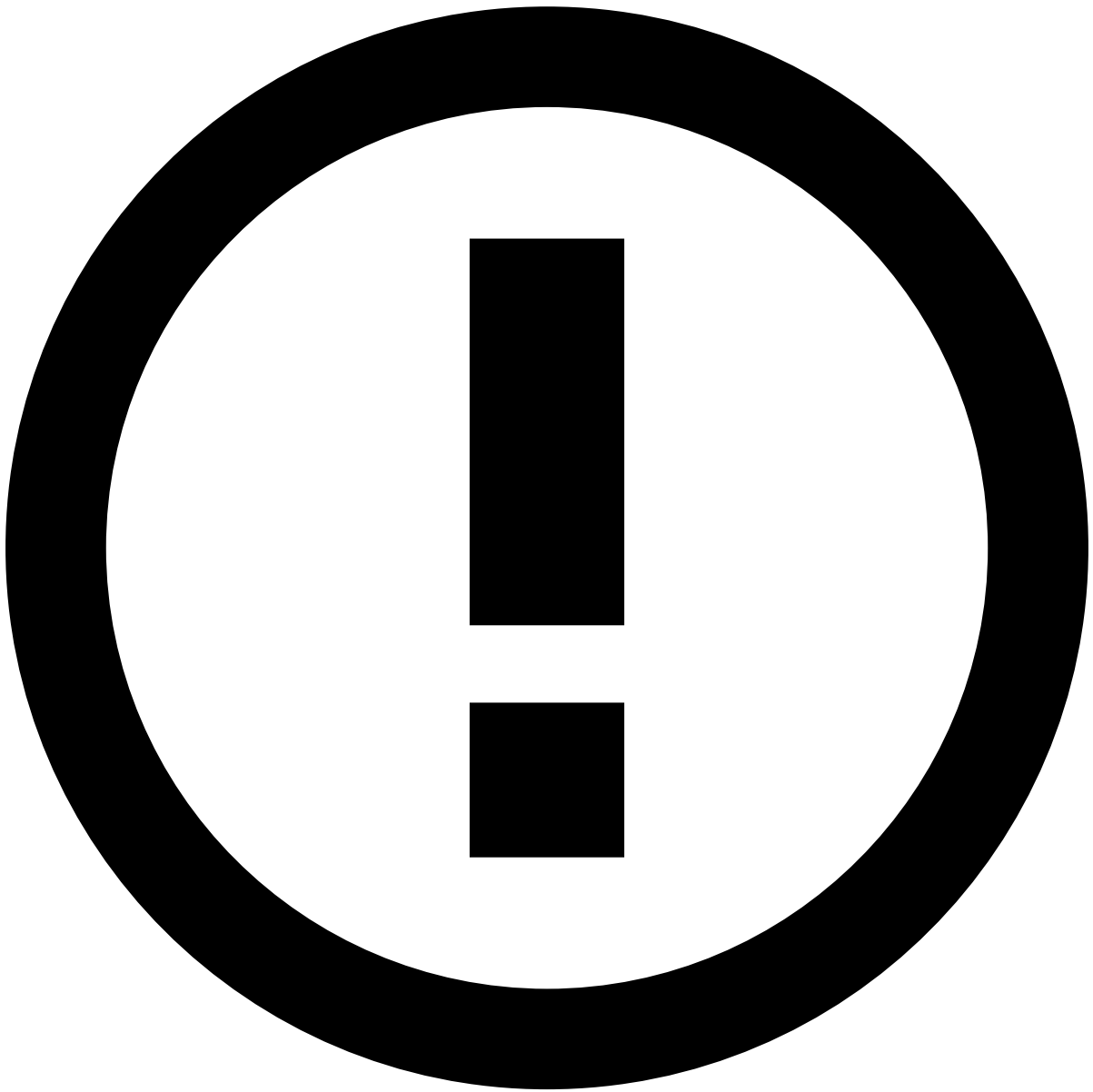
1. In the **Workflows** tab, select an existing **Workflow**.
2. Drag a new **Plan Service** to the **Diagram** canvas.
3.  Edit the **Plan**.
4. In the **Main** tab:
 1. Select the [Data Science Plan](#) you want to apply in your runtime processing. Only **Deployable Plans** are available.
 2. Select the **Plan Execution** to carry the input and output schemas of the **Plan**, as defined during Data Science stage, into your workflow.
5. In the **Schemas** tab:
 1. Confirm that the **Service Input** and **Service Output** are consistent with the data you expect for the **Plan**.
 2. Make sure the **Service Input** and **Service Output** are correctly mapped from the **Event Schema**, for each real-time event moving along the **Workflow**.
6. In the **Properties** tab, set the values for each property to define the service configuration in your **Workflow**. Properties and default values are defined by Pulse.
7.  **Connect** the previous service in the **Workflow** to the **Plan**.

Solving schema conflicts

If Pulse invalidates your [Workflow](#) due to schema mapping conflicts in a Workflow Service, you must solve them as follows:

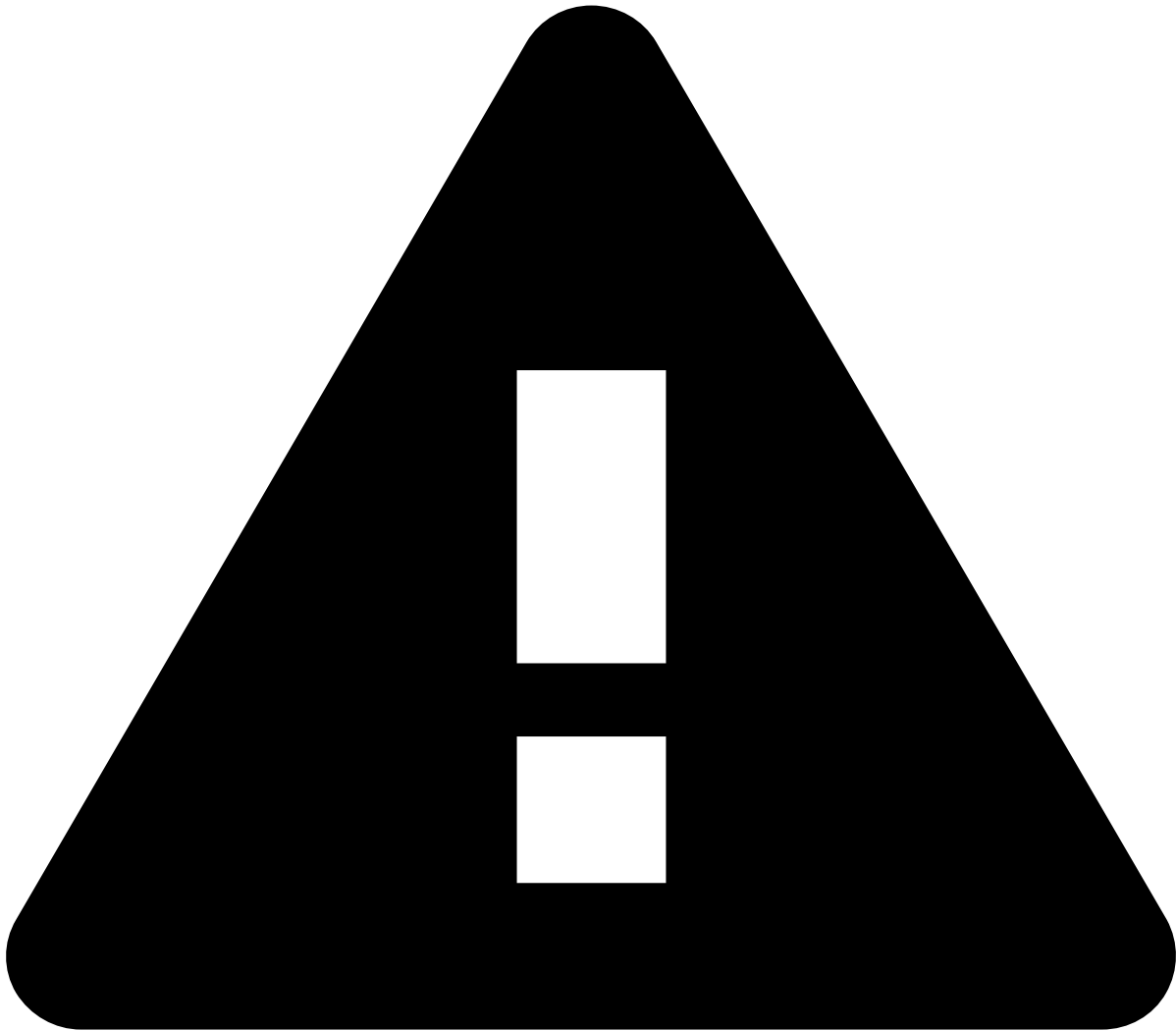
1.  Edit the **Service**.
2. Select the **Schemas** tab.
3. Look for a count of **Input Conflicts** and **Output Conflicts** above 0.
4. Select each **Conflict** filter to see which fields could not be mapped. Alternatively, select the **Service** filter and locate the conflict fields in the **Event Schema**. These are marked as *Missing Input* or *Missing Output*.
5. If the **Event Schema** contains a field that you can map to the **Service Input** or **Output**, manually enter the name of that field.

The following images display schema conflicts during resolution:



Conflicts count

Note that Pulse will still display the original conflicts count even after you have solved them.



Removed fields

If a **Plan** you configured for a **Plan Service** removes fields from the workflow schema, then you must ensure that subsequent services in the **Workflow** do not require those fields. Otherwise, Pulse will generate schema conflicts, invalidate your **Workflow**, and you will have no available fields to map from.

In these cases, you should redesign your **Plan**, articulate it with the remaining Data Science work to be used in runtime, and redeploy it to your **Workflow**.

Addressing errors in plan definition🔗

When defining the components of a **Plan**, you may encounter errors that would result in failures during the [plan execution](#).

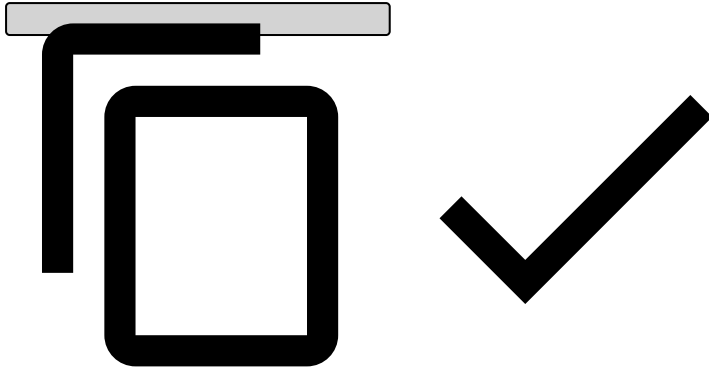
Some of those errors are detectable when defining the Plan, and Pulse will show warnings to allow them to be fixed before deploying the plan to production.

One such error is defining [fields](#) based on a [PQL expression](#) (e.g. within a [Metrics or Map operator](#)). Operators that require null coalescing (i.e. [default fallback values](#)) will produce a warning in the UI if the coalescing operator (`??`) and value are missing:

To fix this situation, provide a fallback value in the PQL expression input field, e.g. `?? 0` or `?? ""`.

A similar warning will be shown if built-in functions are used in collections (like `List` s, `Set` s or `Map` s). For example, an expression like

```
["test", "test1", "test2"].contains("testing")
```

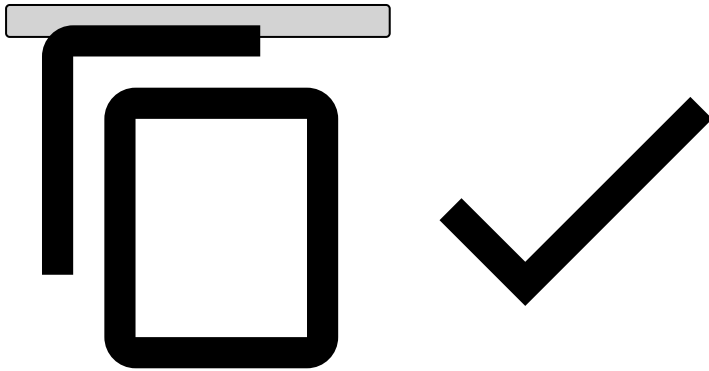


will fail because the `contains` function is not supported in collections used in the PQL fields of Map Operators or Metric Operators.

Attempting to use such functions in that context will result in a warning in the UI:

The alternative is to use [UDFs \(User Defined Functions\)](#), e.g.:

```
com.feedzai.pulse.pkernel.runtime.CollectionUDF.contains(["test", "test1", "test2"], "testing")
```



Besides these preventive warnings, Pulse will also allow recovering from execution errors that are not detectable at design time. Refer to [Automatic Application Repair](#) and [Workflow recovery](#) for more information on how to recover from execution errors.

Next Steps📖

Proceed with the designing of your Workflow by adding, configuring and connecting other services. Check the following pages:

- [Adding input adapters](#)
- [Adding models](#)
- [Adding rules](#)
- [Adding custom services](#)